Technical Report
Point-Supervised Semantic Segmentation on LoveDA

Technical report

1. Introduction and problem

Semantic segmentation normally relies on a dense label for every pixel. Dense annotation of high-resolution remote-sensing imagery is slow and expensive: a single 1024x1024 LoveDA tile contains over one million pixels, and class boundaries (roads, buildings, field edges) are exactly where annotator effort is concentrated. Point supervision asks a cheaper question: what can a segmentation network learn if only *N* pixels per training crop are labelled?

The setting is harder than it first appears. The loss is computed on a tiny, non-uniform sample of the image; most pixels carry no gradient. The sampled pixels are spatially correlated, so a random point set can miss rare classes almost entirely. The annotation budget also has to be honest: if points are redrawn every epoch, the network eventually sees far more than *N* labels per image even though the nominal budget never changes. Finally, evaluation must still use complete masks, otherwise a model can look good simply because it was judged on the same sparse pixels it was trained on.

This report implements the assignment's partial focal loss,

```
L_pFCE = sum_i [ FocalLoss(pred_i, GT_i) * MASK_labeled,i ] / sum_i MASK_labeled,i
```

on LoveDA with a U-Net/ResNet-34 segmentation network, and studies two factors: the number of points per crop (Experiment A) and the point-sampling strategy (Experiment B). All numbers in this report come from the runs in `outputs/`; none are hardcoded.

2. Method

2.1 Dataset and label mapping

LoveDA contains 1024x1024 RGB tiles with single-channel masks and an official train/val split (Rural and Urban domains). The original ids are remapped to a contiguous seven-class problem:

Original id	Class	Training id
0	no-data	255 (`IGNORE_INDEX`)
1	background	0

Original id	Class	Training id
2	building	1
3	road	2
4	water	3
5	barren	4
6	forest	5
7	agriculture	6

No-data pixels are never sampled and never supervised. The full training mask is used only inside the dataset's `__getitem__` to draw points; the item that leaves the method contains the image and a sparse mask in which every non-sampled pixel is 255. Validation uses the complete ground-truth mask.

Subset used here. The experiments use a deterministic, class/domain-balanced subset of the official split: 600 training images (300 Rural, 300 Urban) and 200 validation images (100 Rural, 100 Urban), selected with `dataset.subset_seed=42`. The subset was chosen because the available hardware was a single Tesla T4; the configuration supports the full split by setting `max_train_images` and `max_val_images` to `null`. The exact file list is recorded in `data/LoveDA/subset_manifest.json`.

2.2 Architecture

The network is `smp.Unet(encoder_name="resnet34", encoder_weights="imagenet", in_channels=3, classes=7, activation=None)`: a U-Net decoder with a ResNet-34 encoder pretrained on ImageNet, returning raw logits. It has 24.44M parameters. No YOLO, instance-segmentation, or post-processing components are used.

2.3 Point annotation simulation

For every training crop the pipeline selects exactly N valid pixels and copies their original class ids into a sparse mask; all other pixels become 255.

- **Random sampling** draws N pixels uniformly from the valid pixels of the crop. Its class distribution follows the natural class frequency.
- **Class-balanced sampling** round-robins over the classes present in the crop, so a rare class receives the same number of points as a common one. If a class runs out of pixels, the remaining budget is redistributed to classes that still have pixels.

If a crop contains fewer than N valid pixels, all valid pixels are used and a warning is logged. Sampling is seeded and reproducible.

Fixed point sets. By default the crop window and the point set for a given image are deterministic, seeded by `(seed, image_index)`. The same N pixels are therefore used in every epoch, which makes the annotation budget honest. Geometric augmentation (horizontal/vertical flips and optional 90-degree

rotations) is still random, but it is applied to the image and the sparse mask with identical parameters, so labels stay aligned. Setting `point_sampling.fixed_points=false` redraws both crop and points each epoch; the network then sees more distinct labels over training, and that setting is reported separately rather than mixed into the main results.

2.4 Loss

The per-pixel focal loss is

```
FL = -alpha_t * (1 - p_t)^gamma * log(p_t)
```

where `p_t` is the softmax probability of the ground-truth class. The partial focal loss multiplies `FL` by the labelled mask and divides by the number of labelled pixels, so unlabelled pixels contribute neither to the numerator nor to the denominator. `gamma=0` and `alpha=None` reduces exactly to partial cross-entropy, which is implemented separately as `PartialCrossEntropyLoss` and tested for numerical equivalence.

Implementation details that matter for stability:

- `log_softmax` and the focal term are computed in float32 even under AMP;
- ignored targets are clamped to a safe index before `gather`, then masked out;
- gradients at ignored pixels are exactly zero (unit-tested);
- a batch with no labelled pixels returns a differentiable zero by default, or raises when `loss.strict=true`;
- class weights (`alpha`) are optional and, if used, must be estimated from the sparse labels only.

2.5 Training and evaluation

Setting	Value
Optimizer	AdamW, lr 1e-3, weight decay 1e-4
Schedule	cosine annealing to 1% of the initial lr
Batch size	4
Crop size	512x512 random crop
Epochs	10 (experiments)
Precision	AMP (fp16) on CUDA
Augmentation	horizontal/vertical flips, 90-degree rotations
Normalization	ImageNet mean/std
Validation	deterministic tiled inference, 512x512 tiles, 25% overlap, logit averaging
Metrics	split-level confusion matrix: per-class IoU, mIoU, per-class F1, mean F1, pixel accuracy

Validation accumulates one confusion matrix over the whole split (not a per-image average). Pixels with ground truth 255 are excluded. A class with zero ground-truth pixels in the split is reported as NaN and excluded from mIoU and mean F1. Checkpoints are selected by validation mIoU; `best.pt` and `last.pt` contain weights, optimizer, scheduler, scaler, epoch, resolved config and seed.

Because a 1024x1024 tile does not fit the 512x512 training crop, validation uses sliding-window inference: with 25% overlap the window starts are `[0, 384, 512]` on each axis, i.e. nine 512x512 windows per image, and the logits are averaged in the overlap regions before the argmax. This is deterministic and identical for every run, so the comparisons are not affected by evaluation noise.

What the network actually sees. For each training image the model receives a 512x512 crop with 10-1000 labelled pixels (0.004%-0.4% of the crop) and 255 everywhere else. All reported runs use `fixed_points=true`, so the crop window and the point set are identical in every epoch; only the flips/rotations change.

3. Experiments

Both experiments share the split, architecture, preprocessing, optimizer, schedule, epoch budget and seed (42). The only changed factor is stated in each subsection. Runs are stored under `outputs/exp_density/` and `outputs/exp_sampling/`; `pointseg-aggregate` computes mean and standard deviation over seeds from the saved `metrics.json` files.

4. Discussion

5. Limitations and future work

- **Single seed and a subset.** The reported runs use one seed and 600/200 images on one T4. The runners accept `--seeds` and the config accepts the full split, but multi-seed error bars on the full dataset were outside the available compute. Differences of a few mIoU points should therefore be read as trends, not as tightly estimated effects.
- **No semi-supervised use of unlabelled pixels.** The assignment's larger system includes semi-supervised learning on the unlabelled pixels; this work deliberately isolates the effect of the partial loss and point budget. Pseudo- labelling, consistency regularization, or entropy minimization on the 255 pixels are the natural next step.
- **No test-time augmentation or ensembling.** TTA and model ensembles usually add a few mIoU points but would obscure the comparison of annotation budgets.
- **No histogram matching.** Domain adaptation between Rural and Urban is left out; histogram matching could reduce the domain gap but is a separate factor.

- **Fixed crops.** The default fixed point set also fixes the crop window, so each image contributes one 512x512 region. This is the honest-budget choice; resampling crops and points each epoch is supported and would likely improve raw accuracy at the cost of a larger effective annotation budget.

6. Conclusion

Appendix: reproducibility

- Hardware: one Tesla T4 (15 GB, driver 580.82.07); PyTorch 2.15.0+cu132 (CUDA 13.2, cuDNN 9.26), torchvision 0.29.2, segmentation-models-pytorch 0.6.0, Python 3.13.
- Seed 42 for Python, NumPy and torch; DataLoader workers are seeded.
- Every run writes `config.yaml`, `metadata.json` (library versions and git commit), `history.csv`, `metrics.json`, `best.pt` and `last.pt`.
- Dataset files, checkpoints and outputs are git-ignored; the subset manifest is kept with the data.
- Commands:

```
uv sync --group dev
uv run pytest
uv run pointseg-check-data --config configs/default.yaml
uv run pointseg-exp-density  --config configs/exp_density/A3.yaml
uv run pointseg-exp-sampling --config configs/exp_sampling/random.yaml
uv run pointseg-aggregate    --root outputs
```